Design Decisions — Intelligent University Course Finder

---

1. Retrieval Strategy — Hybrid Search over Pure Semantic or Keyword

Decision: Combine **Qdrant dense semantic search** with **BM25 keyword search**, fused using **Reciprocal Rank Fusion (RRF)**.

Why not pure semantic? Dense vector search excels at finding conceptually related content but can miss exact keyword matches. A query for *"Python for data science"* might return a ML course with no Python mention, while the intended Python-specific course ranks lower.

Why not pure BM25? Keyword search misses paraphrasing. A student typing *"machines that learn automatically"* would get zero matches for a course titled *"Introduction to Machine Learning"*.

Why RRF over weighted average? RRF is parameter-free — it doesn't require tuning blend weights. It rewards consistently high-ranked results across both retrievers, which is robust without a training set.

Why Small-to-Big Parent Fetch? Chunking long course descriptions into smaller pieces improves retrieval precision. But returning the raw chunk lacks context. Parent fetch retrieves the full description after the child chunk is matched, giving agents richer context without increasing embedding noise.

1b. Chunking Strategy — Hybrid Parent/Child with 3 Child Types

Decision: Each course is indexed as **1 parent document + 3 specialised child documents + 1 BM25 document**.

Why not chunk by fixed token size?

Fixed-size chunking is domain-agnostic and cuts mid-sentence. Course data has clearly defined semantic fields (title, skills, description). Field-aware chunking produces cleaner, more meaningful embedding representations.

Why a parent/child split?

Layer	Purpose
Parent	Full composite text (title + org + skills + description). Returned to the agent for rich context after retrieval. Never directly embedded for search.
Child	Small, focused field chunks. Embedded and searched. High retrieval precision because each chunk carries a single semantic signal.

The 3 child document types

Child Type	Content	Why
Identity	`title + organization + difficulty + rating`	Captures "what course is this and who offers it" — anchors semantic search to course identity
Skills	`skills list`	Enables skill-to-skill matching — "I want to learn Python" matches courses listing Python as a skill
Description	`what you'll learn + course description`	Captures deeper conceptual meaning — "understand how machines learn automatically" finds ML courses even without exact keywords

Why 3 child chunks per course?

A single chunk per course forces a trade-off: embed the skills text and lose description semantics, or embed the description and lose skill-level precision. Three purpose-built chunks let Qdrant find the right course via whichever signal is strongest in the query.

BM25 document

A single keyword-optimised concatenation of all textual fields per course. Used by the BM25 retriever independently. Ensures exact keyword matches (e.g. course codes, specific tool names)

are never missed by the semantic model.

---

2. Agent Framework — OpenAI Agents SDK over LangChain Agents

Decision: Use the **OpenAI Agents SDK** (`agents.Agent`, `Runner`, `@function_tool`) for all LLM agents.

Why OpenAI Agents SDK? - Built-in tool-calling loop — no manual parsing of function calls - Native `@function_tool` decorator infers schemas from type hints and docstrings automatically - `Runner.run()` handles multi-turn reasoning, retries, and output extraction cleanly - Matches the established reference project pattern (Day_29 ecommerce assistant)

Why not LangChain AgentExecutor? LangChain agents are verbose, require manual chain construction, and have a steeper debugging surface. The SDK provides cleaner structured tool output directly.

Why use LangChain at all then? LangChain is used **only in guardrails** for its `PydanticOutputParser` — a convenient way to enforce structured JSON from a prompt without building agents. It is not used in the main agent pipeline.

---

3. Agent Pipeline Design — Deterministic Coordinator over Orchestrator Agent

Decision: Use a **hardcoded Python pipeline** (`recommend_logic.py`) instead of an LLM-based orchestrator agent that decides which agents to call.

Why? The pipeline flow is always the same: Intent → Skill Gap + Career + Sequencer → Advisor. An LLM orchestrator adds: - Unpredictability (might skip agents) - Extra latency (one more LLM call before any work starts) - Extra API cost

A deterministic coordinator is faster, cheaper, and 100% reliable.

When would an orchestrator make sense? If the system needed to support multiple query types (e.g. "compare two courses", "find a mentor", "get a syllabus"), an LLM orchestrator would route to the right specialist agent. This is a planned future enhancement.

---

4. Parallelism — asyncio.gather for Step 4

Decision: Run `SkillGapAgent` and `CareerAgent` simultaneously using **`asyncio.gather`**.

Why? Both agents receive the same input (the top 10 courses) and produce independent outputs. There is no data dependency between them. Running them in parallel cuts Step 4 from ~2× latency to ~1× latency.

Why not parallel with IntentAgent? The IntentAgent must complete first because its output (the 10 retrieved courses) is the input to all Step 4 agents. Sequential dependency here is unavoidable.

CourseSequencer is sync: Pure Python sorting — no LLM call, no I/O wait. It runs inside the same event loop iteration at effectively zero cost.

---

5. Token Optimization — Slimmed Course Representation

Decision: Return only 6 fields per course (`course_id`, `course_name`, `organization`, `difficulty`, `rating`, `skills`, `course_url`) to downstream agents, truncating `skills` to 100 characters.

Why? The raw course object from Qdrant includes full descriptions, page content, chunk metadata, and embeddings — often >2000 tokens per course. Passing 10 full records to every downstream agent would cost thousands of tokens per request.

The slim representation gives agents everything they need: - `course_name` + `skills` for understanding course content - `difficulty` for sequencing and gap analysis - `rating` for quality-aware sorting - `course_url` for the final frontend link

---

6. PII Handling — Presidio over LLM-based Redaction

Decision: Use **Microsoft Presidio** (rule-based + NLP) for PII detection and redaction, not the LLM.

Why not use the LLM for PII? Asking GPT-4o to redact PII means sending the PII to OpenAI's API first, which defeats the purpose. Presidio runs **locally on CPU** using spaCy. No PII ever leaves the local machine.

What Presidio detects: - Person names (`<PERSON>`) - Email addresses (`<EMAIL_ADDRESS>`) - Phone numbers (`<PHONE_NUMBER>`) - Location/addresses (`<LOCATION>`) - Credit card numbers, IDs, etc.

The LangChain guardrail runs after Presidio: Once PII is removed, the now-safe query is sent to the LLM purely to validate that it's a genuine learning intent — not spam, toxic content, or a prompt injection attack.

---

7. API Architecture — Layered FastAPI over Monolithic

Decision: Structure the FastAPI backend into four distinct layers: `routes/`, `core/`, `utils/`, and `schemas/`.

Layer	Responsibility
`routes/`	HTTP endpoint definitions, request/response binding
`core/`	Business logic — agent orchestration, pipeline execution
`utils/`	Pure helper functions (data transformations, formatting)
`schemas/`	Pydantic models for all I/O contracts

Why? Separation of concerns. Routes should not contain business logic. Logic should not contain HTTP concerns. This makes each layer independently testable and replaceable.

---

8. API Gateway — Separate Proxy Service

Decision: Add a dedicated **API Gateway** (`api_gateway/gateway.py`) on port 8080 as the single entry point, proxying to the backend microservice on port 8000.

Why? - **Decoupling:** The frontend never directly knows the backend's address or port - **Single entry point:** Future features (auth, rate limiting, routing to multiple microservices) are added here without touching backend logic - **Error handling:** The gateway catches connection errors and timeouts before they reach the frontend, returning clean 503/504 responses

Technology: Uses `httpx` for async HTTP proxying — lightweight, no additional infrastructure (e.g. no Nginx, Kong, or AWS API Gateway needed for development).

---

9. Course Sequencing — Rule-Based over LLM

Decision: Order courses by difficulty using **Python sorting** (Beginner → Intermediate → Advanced → Mixed), not an LLM.

Why? Difficulty ordering is deterministic — there is no ambiguity. Routing this through an LLM would: - Add ~2–3s of latency - Cost tokens - Risk non-deterministic ordering

Within the same difficulty tier, courses are sorted by `rating` descending — highest-rated beginner courses appear first.

---

10. Frontend Architecture — Reusable Component Model

Decision: Structure the React frontend around **4 single-responsibility components**: `SearchBar`, `CareerCard`, `AdvisorCard`, `CourseList`.

Component	Renders
`SearchBar`	Query input + loading state
`CareerCard`	Career track + alignment reason
`AdvisorCard`	Advisor summary, study tips, skill gap tags
`CourseList`	Difficulty-staged pathway with course cards

Why? Each component maps to exactly one agent's output. Adding a new agent in the future means adding one new component — no other component needs to change.

Styling: Pure CSS with CSS variables for the design system — no Tailwind. Minimalist light theme using solid flat panels, 1px soft borders, clean typography, and subtle flat interactions.

---

11. Logging — Structured Per-Module Logger

Decision: Create a **`get_logger(name)`** factory in `logger.py` that writes structured timestamped logs to both the console and a per-module `.log` file under `logs/`.

Why? With 5 LLM calls per request, tracing which agent did what and when is critical for debugging. Per-module log files (`logs/intent_agent.log`, `logs/guardrails.log`, etc.) allow isolated inspection of each agent's behaviour without noise from others.

---

12. Environment Configuration — `.env` Loaded Before SDK Import

Decision: Call `load_dotenv()` **before** any `from agents import ...` or `from langchain_openai import ...` import.

Why? The OpenAI Agents SDK initialises its trace exporter at **import time** and reads `OPENAI_API_KEY` from `os.environ` in that moment. If `load_dotenv()` runs after the import, the trace exporter sees an empty key and logs `"skipping trace export"` — even though the key exists in `.env`. Loading the env first ensures the key is in `os.environ` before any SDK code runs.